

Contents

Guide 9 — Authentication & Admin, the simple version	1
1. The idea, in three sentences	1
2. How login works	1
3. Roles and how someone becomes an admin	1
4. Why these choices	1
5. The files	2
6. Jury questions — ready answers	2

Guide 9 — Authentication & Admin, the simple version

How Mobile Match Algeria handles accounts, login, and the admin role — in plain words, with a worked flow, the files, and ready answers for the jury.

1. The idea, in three sentences

Visitors can browse freely, but to save plans, set a profile, or get recommendations they create an account and log in. Authentication is handled by **NextAuth (Auth.js)** with email + password, passwords are stored **hashed with bcrypt** (never in plain text), and a **role** (USER or ADMIN) decides who can reach the admin tools.

2. How login works

1. **Register** — the user submits name, email, password. The input is validated (zod), the request is rate-limited, and the password is **hashed with bcrypt** (cost 12) before the account is stored. The plain password is never saved.
 2. **Log in** — NextAuth's Credentials provider looks up the user by email and **bcrypt-compares** the typed password against the stored hash. On success it issues a **JWT session** (a signed token), not a server-side session row.
 3. **Carry identity** — the token holds the user's `id` and `role`, so every request knows who the user is and what they're allowed to do.
 4. **Log out** — clears the session.
-

3. Roles and how someone becomes an admin

- Two roles: **USER** (the default for every new account) and **ADMIN**.
 - There is **no public admin sign-up**. A logged-in user becomes an admin only by entering a secret access code on the claim endpoint (`/api/admin/claim`): if the code matches the server's `ADMIN_ACCESS_CODE`, their role is updated to ADMIN. Without the code, no promotion happens.
 - Admin-only pages and endpoints check the `role` carried in the signed token on the server, so a normal user cannot reach them by guessing a URL.
 - Admins get the **admin dashboard**: platform stats, operator health, scrape history, a "Run scrape" button, and the admin notification stream.
-

4. Why these choices

- **NextAuth / Auth.js** — a maintained, audited library; you should never hand-roll session handling and token signing.
 - **JWT sessions** — stateless, so no database lookup is needed on every request.
 - **bcrypt** — the industry-standard password hash: salted and deliberately slow, which makes stolen hashes expensive to crack.
-

5. The files

- **lib/auth.ts** — the NextAuth config: Credentials provider, bcrypt check, JWT + role callbacks.
 - **api/auth/[...nextauth]/route.ts** — the login / session / logout endpoints.
 - **api/auth/register/route.ts** — sign-up (zod + rate limit + bcrypt hash).
 - **api/admin/claim/route.ts** — promote to admin via the access code.
 - **login/page.tsx** and **admin/page.tsx** — the login screen and the admin dashboard.
-

6. Jury questions — ready answers

- **How are passwords stored?** Hashed with bcrypt (cost 12), salted; the plain password is never stored or logged.
 - **Why NextAuth instead of your own login?** Authentication is easy to get subtly wrong; using a maintained library avoids whole classes of bugs.
 - **How does someone become an admin?** Only by entering the secret access code at the claim endpoint, which sets their role server-side. There is no public admin registration.
 - **Can a normal user fake admin access?** No — the role lives in the signed JWT and is checked on the server; tampering invalidates the token.
 - **Session type?** A stateless JWT, so requests don't hit the database just to authenticate.
-

Companion to the security guide (validation, rate limiting, access control).